



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A' Grade

Department of Computer Science and Engineering DevOps Lab - L3

Lab Assignment No: 8	
Name of Student	Aryan Sheladia
PRN	23070122202
Batch	2023 - 2027
Class	CSE – B1
Academic Year & Semester	2026 - 2027, 7th Semester
Title of Assignment	Dockerfile & Image build
Implementation & Theory	Assignment - 8 >  app.py > ... 1 from flask import Flask 2 3 app = Flask(__name__) 4 5 @app.route("/") 6 def home(): 7 return "Hello, Docker!" 8 9 if __name__ == "__main__": 10 app.run(host="0.0.0.0", port=5000) app.py created — Wrote a minimal Flask application with a single / route returning "Hello, Docker!". The application is bound to 0.0.0.0 so that it can be accessed from outside the Docker container. Assignment - 8 >  requirements.txt 1 Flask==2.1.2 2 Werkzeug==2.1.2 requirements.txt created — Specified the Flask application dependencies required to run the application inside the Docker container. The required Flask and Werkzeug versions were pinned to

ensure compatibility.

Assignment - 8 > Dockerfile > ...

```
1 FROM python:3.9-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 COPY app.py .
10
11 EXPOSE 5000
12
13 CMD ["python", "app.py"]
```

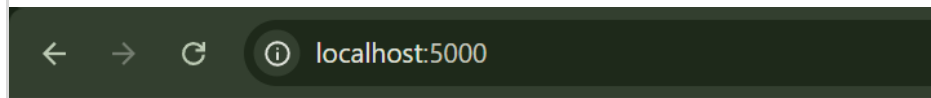
Dockerfile created — Defined the Docker image configuration using a Python base image. The Dockerfile sets the working directory, installs dependencies from requirements.txt, copies the Flask application, exposes port 5000, and specifies the command to start the application.

```
PS E:\DevOps_SEM_7\Assignment - 8> docker build -t my-flask-app .
=> => transferring dockerfile: 216B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aabb1acda1e13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aabb1acda1e13cf1731b1b
=> => sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c450078315beb508 251B / 251B
=> => sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08 13.88MB / 13.88MB
=> => sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d 1.29MB / 1.29MB
=> => sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d 29.78MB / 29.78MB
=> => extracting sha256:38513bd7256313495cdd83b3b0915a633cfa475dc2a07072ab2c8d191020ca5d
=> => extracting sha256:b3ec39b36ae8c03a3e09854de4ec4aa08381dfed84a9daa075048c2e3df3881d
=> => extracting sha256:fc74430849022d13b0d44b8969a953f842f59c6e9d1a0c2c83d710affa286c08
=> => extracting sha256:ea56f685404adf81680322f152d2cfec62115b30dda481c2c450078315beb508
=> [internal] load build context
=> => transferring context: 288B
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY app.py .
=> exporting to image
```

Docker image build — Built the Docker image my-flask-app using the Dockerfile. Docker processed the Python base image, installed the Flask dependencies from requirements.txt, copied the application code, and successfully created the image.

```
PS E:\DevOps_SEM_7\Assignment - 8> docker run -d -p 5000:5000 --name my-flask-container my-flask-app
48d66661140e4921a8077f2d8aadd1b783f8d91dfa653d67350ec06f43c56606
PS E:\DevOps_SEM_7\Assignment - 8> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
48d66661140e   my-flask-app   "python app.py"         16 seconds ago Up 16 seconds  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   my-flask-container
PS E:\DevOps_SEM_7\Assignment - 8>
```

Docker container started — Created and started a container from the my-flask-app image. Port 5000 of the container was mapped to port 5000 of the host, allowing the Flask application to be accessed from the host machine.



Hello, Docker!

Flask application verified — Accessed the containerized Flask application through the browser using localhost:5000. The successful display of **"Hello, Docker!"** confirms that the Docker container is running correctly and the Flask application is accessible through the mapped port.